

SELECTED ENGINEERING WORK

VARUN VIJAY

Embedded Systems & Software Engineering Portfolio

Computer Engineering student focused on embedded firmware, distributed robotic control, vehicle electronics, hardware-software integration, and practical debugging at the signal level.

STM32 / Embedded C

CAN / PWM / ADC

ROS2 / Linux

Python / C++

Oscilloscope Debugging

KiCad / SOLIDWORKS

FOCUS AREAS

Embedded Firmware • Robotics • Software

A reusable portfolio of selected technical work for firmware, embedded systems, robotics, electronics, and software engineering opportunities.

University of Waterloo - BAsC Computer Engineering

Email: v5vijay@uwaterloo.ca

Phone: 226-339-1729

LinkedIn: [linkedin.com/in/varun-vijay](https://www.linkedin.com/in/varun-vijay)

GitHub: github.com/varunvj2006



GitHub



LinkedIn



Email

ENGINEERING PROFILE

Building dependable systems across firmware, electronics, and software

My strongest work sits at the boundary between code and hardware: sensing real signals, controlling physical systems, moving data between processors, and debugging timing or communication issues with lab equipment.

WHAT I BRING

- **Embedded firmware:** STM32F401 development in C, PWM generation, encoder feedback, ADC-based sensing, servo control, and CAN communication.
- **Robotics and distributed systems:** ROS-based architecture on Raspberry Pi 5 and Ubuntu Linux, connected to low-level motor control over serial and Ethernet interfaces.
- **Hardware integration:** power-distribution integration, current sensing, emergency shutdown controls, indicators, and practical validation using oscilloscopes and function generators.
- **Software depth:** Python data pipelines, semantic filtering, desktop monitoring tools, web APIs, C/C++, Git, Docker, and Linux workflows.

2

STM32 controllers integrated over CAN

25%

faster search response in Sniffd AI

3+

physical systems built or integrated

CORE TECHNICAL STACK

LANGUAGES

C, C++, Python, JavaScript, MATLAB, VHDL, Verilog, RISC-V Assembly

PLATFORMS

ROS2, Raspberry Pi 5, Ubuntu Linux, Docker, Git

DESIGN

SOLIDWORKS, AutoCAD, Figma

EMBEDDED

STM32Cube IDE, STM32 HAL, CAN, PWM, ADC, encoders, sensor integration

HARDWARE TOOLS

Oscilloscope, function generator, digital multimeter, KiCad

SOFTWARE

Requests, BeautifulSoup, Tkinter, React, REST APIs, JSONL pipelines

EDUCATION

University of Waterloo

Bachelor of Applied Science, Computer Engineering
September 2024 - June 2029

Presidential Scholarship of Distinction

Relevant coursework: Digital Computers, Digital Circuits and Systems, Algorithms and Data Structures, Electronic Circuits, Embedded Microprocessor Systems, Signals and Systems, Systems Programming and Concurrency.

ROBOTICS RESEARCH ASSISTANT - REAL-TIME EMBEDDED SOFTWARE GROUP

BenchBot distributed control platform

A ROS-based robotics architecture linking high-level control on a Raspberry Pi 5 to STM32F401 motor and servo firmware.

STM32F401

Embedded C

ROS

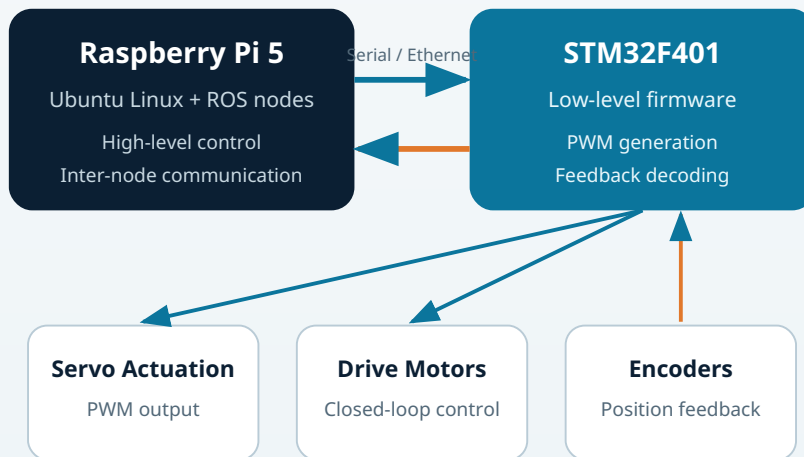
Raspberry Pi 5

Ubuntu 22.04

PWM

Quadrature Encoders

System architecture



Validated timing and signal integrity with oscilloscopes and function generators

MY CONTRIBUTION

- Created a distributed ROS control architecture on Raspberry Pi 5 running Ubuntu 22.04 LTS.
- Programmed STM32F401 firmware to generate precise PWM signals for servo actuation.
- Integrated quadrature encoder feedback for closed-loop motion control and real-time position tracking.
- Developed signal-decoding routines for servo feedback data.
- Connected high-level and low-level modules through serial and Ethernet interfaces.

Debugging focus: I used function generators and oscilloscopes to investigate communication timing and signal-integrity issues rather than treating the firmware as a black box.

Transferable engineering value

The project required deterministic low-level control, dependable feedback, clear separation between control layers, and disciplined hardware validation - skills that transfer directly to robotics, industrial automation, mobility, aerospace, and other real-world systems.

EMBEDDED SYSTEMS DEVELOPER - ELECTRIUM MOBILITY DESIGN TEAM

Dual-STM32 CAN vehicle control system

A distributed embedded architecture for an electric human-powered vehicle, separating power and speed control from user interface and system feedback.

Dual STM32

CAN Bus

Hall-effect Current Sensor

Fault Handling

KiCad

Power Distribution

SYSTEM RESPONSIBILITIES

Controller A - power and speed

Handles the vehicle-side sensing and control path, including battery and power-monitoring data.

Controller B - user I/O and feedback

Handles user-facing commands, status outputs, system feedback, LEDs, and buzzer behaviour.

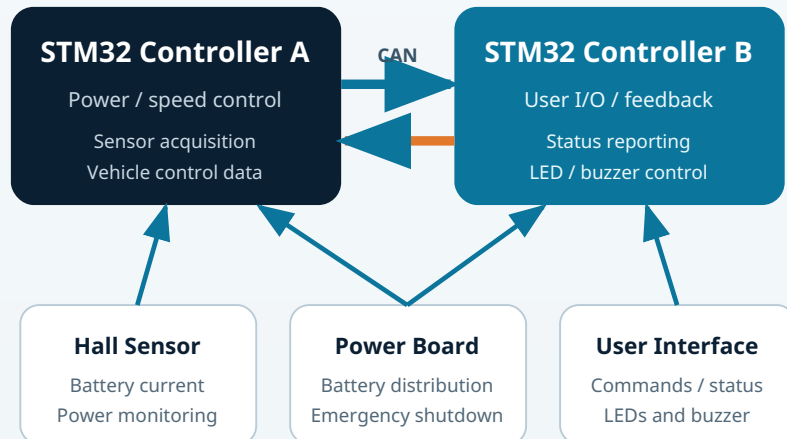
Safety and power integration

Supports MCU-controlled emergency shutdown and fault indication through a custom power-distribution design.

MY CONTRIBUTION

- Developed embedded firmware for two STM32 microcontrollers.
- Configured CAN communication to transfer sensor, control, and status data.
- Integrated Hall-effect current-sensor readings for battery and power monitoring.
- Supported KiCad-based power-distribution integration with shutdown, LED, and buzzer outputs.

Vehicle electronics architecture

**Fault response path**

Sensor/control status - CAN coordination - emergency shutdown
LED and buzzer alerts communicate system state to the user

AI SECURITY RESEARCH ENGINEER - REAL-TIME EMBEDDED SOFTWARE GROUP

Semantic cybersecurity data pipeline

A Tor-enabled Python crawler and semantic filtering workflow for collecting, ranking, reviewing, and storing cybersecurity-relevant information from decentralized sources.

Python

Tor Networking

Requests

BeautifulSoup

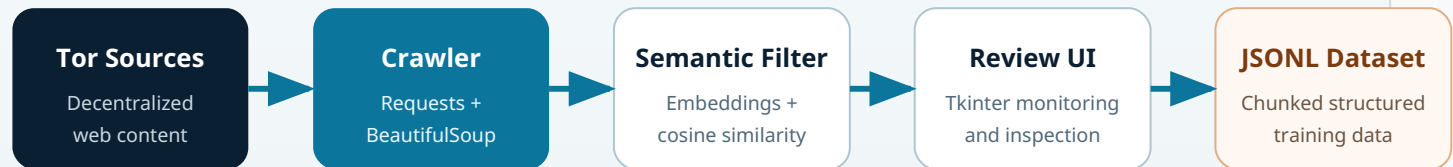
Text Embeddings

Cosine Similarity

Tkinter

JSONL

Research pipeline



ENGINEERING CONTRIBUTION

- Designed and implemented a Tor-based semantic web crawler using Python, Requests, and BeautifulSoup.
- Built a custom embedding and cosine-similarity pipeline to automatically identify security-relevant material.
- Created chunked processing and JSONL storage for downstream AI analysis and model-training workflows.
- Developed a Tkinter interface for real-time execution monitoring and inspection.
- Applied IoT security, firmware-analysis, and vulnerability-research concepts to guide semantic modelling.

Software engineering habits

Modular pipeline design, explicit data interfaces, observable runtime behaviour, structured outputs, and iteration based on real collected data.

Security perspective

The work connects embedded-system awareness with data acquisition and AI tooling, including attention to firmware analysis, IoT security, and vulnerability research.

Transferable value

This project demonstrates that I can work beyond bare-metal code: I can also build the software tooling, data pipelines, and interfaces used to support testing and engineering decisions.

ADDITIONAL WORK

End-to-end embedded and software projects

These projects show complete implementation across sensing, control logic, user feedback, physical design, and application-level software.

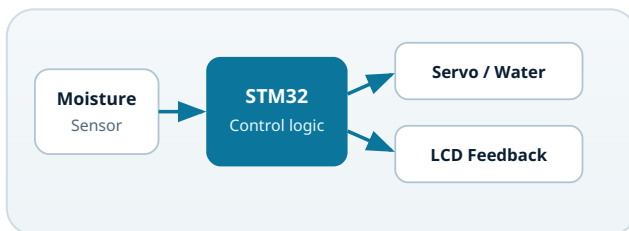
EMBEDDED SYSTEM

Automated Plant Watering System

STM32 C ADC Sensing Servo LCD

SOLIDWORKS

- Programmed an STM32 microcontroller in C to automate soil-moisture-based water delivery.
- Integrated sensor acquisition, servo control, and an LCD status display.
- Calibrated and troubleshot the sensor-hardware-software interface to improve watering accuracy and reliability.
- Designed and produced simple 3D-printed mechanical components in SOLIDWORKS.

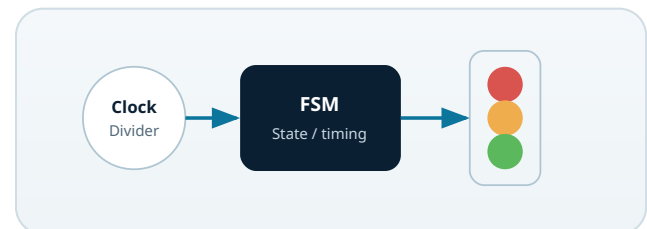


DIGITAL DESIGN

FPGA Traffic Light Controller

VHDL Quartus FPGA Finite-State Machine

- Designed a traffic-light controller in VHDL using a finite-state-machine architecture.
- Implemented clock dividers, state counters, and timed LED sequencing.
- Validated behaviour through Quartus simulation and on-board FPGA debugging.
- Used seven-segment displays and hardware timing analysis to inspect state and timing behaviour.



APPLICATION SOFTWARE

Sniffd AI

A functional fragrance-recommendation prototype that converts natural-language descriptions into structured olfactory notes, retrieves related products through Google Search API integration, and uses Ollama/Python for real-time matching.

25%

search-response improvement through query caching and request handling

Python Ollama Google API JavaScript
HTML/CSS

Let's build dependable systems.

Available for Fall 2026 co-op opportunities in firmware, embedded systems, robotics, electronics, and software engineering.

v5vijay@uwaterloo.ca

226-339-1729

github.com/

varunvj2006